



3D-power 图的快速生成方法

桂志强, 姚裕友, 张高峰, 徐本柱, 郑利平*
(合肥工业大学 计算机与信息学院, 安徽 合肥 230009)

摘 要: 3D-power 图在图形学和流体仿真等领域应用广泛。为解决已有的 3D-power 图计算方法时间性能较差的问题, 提出了基于 GPU 的 power 图构造算法, 给出了一种用于计算 power 图各区域之间的面积估值方法, 使基于 GPU 的构造算法与 Lloyd 算法、牛顿法相结合, 生成满足约束条件的 3D 质心容量限制 power 图 (3D-centroidal capacity constrained power diagram, 3D-CCCPD)。结果表明, 本文算法的时间性能较已有的 3D-power 图构造方法提高了几个数量级。

关 键 词: 3D-power 图; GPU 加速; 质心; 容量

中图分类号: TP 391.41

文献标志码: A

文章编号: 1008-9497(2021)04-410-08

GUI Zhiqiang, YAO Yuyou, ZHANG Gaofeng, XU Benzhu, ZHENG Liping (School of Computer Science and Information Engineering, Hefei University of Technology, Hefei 230009, China)

An efficient computation method of 3D-power diagram. Journal of Zhejiang University (Science Edition), 2021, 48(4):410-417

Abstract: 3D-power diagrams have a wide range of applications in the fields of graphics and fluid simulation. To overcome the poor time performance of the existing 3D-power diagram construction algorithms, a novel GPU-based construction algorithm is proposed. We introduce an estimation algorithm for computing the area between power cells. The estimation algorithm enables the construction algorithm to be coupled with the Lloyd algorithm and Newton's algorithm to generate a 3D-centroidal capacity constrained power diagram(3D-CCCPD). Experiment results show that our approach raises the efficiency of the 3D-power diagram construction up to several orders of magnitude.

Key Words: 3D-power diagram; GPU accelerate; centroidal; capacity

0 引 言

Voronoi 图在日常生活和自然界中普遍存在, 因其优良的几何特性, 被广泛应用于可视化、几何建模、建造设计等领域。power 图是 Voronoi 图的扩展, 即对 Voronoi 图站点引入权重概念, 并重新定义距离。对 power 图的区域施加容量限制, 得到容量限制 power 图(capacity constrained power diagram, CCPD)。进一步对 CCPD 站点施加质心限制, 得到

质心容量限制 power 图 (centroidal capacity constrained power diagram, CCCPD)。

Power 图因其优良的特性被广泛应用于 2D 平面中的采样和点画^[1]、蓝噪声^[2-4]、计算机动画^[5]、3D 空间流体仿真^[6-8]等。

AURENHAMMER^[9-10]提出 power 图的概念, 并对 power 图的性质、应用进行了总结, 提出用分治法构建 power 图; BALZER 等提出用“试位法”优化容量约束, 给出有限区域^[11]和连续区域^[12]的 CCPD 算法, 但时间复杂度较高, 且收敛性差;

收稿日期: 2020-09-23.

基金项目: 国家自然科学基金资助项目(61972128, 61702155); 安徽省自然科学基金资助项目(1808085MF176).

作者简介: 桂志强(1995—), ORCID: <https://orcid.org/0000-0002-9071-1070>, 男, 硕士, 主要从事计算机图形学与计算机辅助设计研究.

*通信作者, ORCID: <https://orcid.org/0000-0001-5071-9628>, E-mail: zhenglip@hfut.edu.cn.

GOES 等^[2]提出用牛顿法优化权重,并结合 MULLEN 等^[13]提出的自适应步长梯度下降法优化质心位置,二者交替迭代,生成 CCCPD,但优化过程中存在相互干扰,收敛速度较慢;XIN 等^[14]提出一种超线性收敛算法,将 L-BFGS 与牛顿法相结合,生成 CCCPD。

相对 2D 领域,3D-power 图的生成复杂性大幅提升,目前大部分研究聚焦于 3D-Voronoi 图^[15-16],并未将其推广至 power 图领域,其可能性和适应性也未得到充分证明;YAN 等^[17]提出了一种以计算几何算法库 (computational geometry algorithms library, CGAL)^[18]为基础,用边界剪裁方法计算 3D-Voronoi 图的算法;RAY 等^[19]提出了无网格 3D-Voronoi 图的图形处理器 (graphics processing unit, GPU) 算法;LIU 等^[20]在 YAN 等^[17]的基础上提出了基于 GPU 的 3D-Voronoi 图计算算法。

GOES 等^[6]实现了基于 VORO++^[21]的线程安全的并行 power 图构造算法,计算效率得到较大提高;ZHAI 等^[8]根据 VORO++ 提出了一种基于 GPU 的裁切并行算法。这些算法适用于站点分布较为均匀的情况,如流体仿真的 power 图,对随机分布的站点表现不佳。GOLIN 等^[22]证明了在 N 个随机站点中 3D-Voronoi 图的复杂度 (即顶点、边和面的数量) 达到 $O(n^2)$,而如果站点均匀独立分布,则复杂度可降至 $O(n)$ 。对于随机分布的站点,目前比较稳定和有效的是基于 CGAL 和 VORO++ 的 power 图构造方法。

RONG 等^[23-24]提出了基于 GPU 的跳转泛洪算法 (jump flooding algorithm, JFA) 构造 Voronoi 图;ZHENG 等^[25]将其扩展至 2D 平面 power 图,用连通域算法提取 power 图的顶点和边以优化生成 CCCPD,算法的时间效率较高,可扩展至 3D 空间,但因 3D-power 图远较 2D 平面图复杂,连通域算法并不适合计算 3D-power 图中的顶点、边和面。因此,本文以 ZHENG 等^[25]提出的基于 GPU 的构造算法为基础,将其扩展至 3D 空间,并提出了一种新的估值算法,以优化生成 3D-CCCPD。

本文的主要贡献:

(1) 将 JFA 扩展应用于 3D 空间 power 图的生成。

(2) 提出一种估值算法,计算离散 3D-power 图中各区域的面积,与 Lloyd 算法和牛顿法结合优化

生成 3D-CCCPD。

2 相关工作

2.1 Voronoi 图与 power 图

定义 Voronoi 图在域 $\Omega \in E^d$ 内,基于直线距离 $d(x, x_i) = \|x - x_i\|$ 对站点集 $X = \{x_i | x_i \in \Omega, i = 1, 2, \dots, n\}$ 进行区域划分。每个 Voronoi 区域 v_i 定义为

$$v_i = \{x \in v_i | \|x - x_i\| \leq \|x - x_j\|, i = 1, 2, \dots, n, i \neq j\}。 \quad (1)$$

Power 图作为 Voronoi 图的扩展,为每个 Voronoi 站点 x_i 赋予权重 w_i ,本文用欧氏距离平方重新定义距离 d :

$$d(x, x_i) = \|x - x_i\|^2 - w_i。 \quad (2)$$

对于新站点集 $(X, W) = \{(x, x_i) | i = 1, 2, \dots, n\}$,将在域 $\Omega \in E^d$ 内的 power 划分定义为

$$P_i^{w_i} = \{x \in P_i^{w_i} | d(x, x_i) \leq d(x, x_j), i = 1, 2, \dots, n, i \neq j\}。 \quad (3)$$

在 2D 平面,站点 x_i 和 x_j 的 power 区域为凸多边形, $P_i^{w_i} \cap P_j^{w_j}$ 为线段或点;而在 3D 空间, power 区域为凸多面体, $P_i^{w_i} \cap P_j^{w_j}$ 为平面、线段或点。如图 1 所示,定义 e_{ij} 为相邻站点 x_i 和 x_j 间的距离, A_{ij} 为 $P_i^{w_i} \cap P_j^{w_j}$ 的面积。对 power 区域 $P_i^{w_i}$,有

$$P_i^{w_i} \cap P_j^{w_j} = \emptyset, i \neq j, \bigcup_{i=1}^n P_i^{w_i} = \Omega。$$

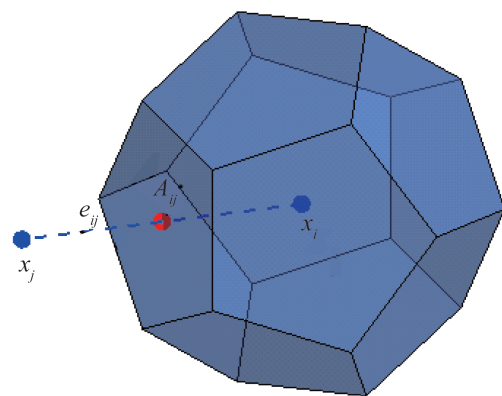


图1 3D-power 区域

Fig.1 3D-power cell

值得注意的是,当各站点权重相等时, power 图便退化为 Voronoi 图^[9]。

2.2 质心容量限制 power 图

引入站点权重,使得 power 图的容量可控。对

其施加精确容量限制,即可得容量限制 power 图。设在问题域 $\Omega \in E^d, d=3$ 中,存在连续密度函数 $\rho(x)$,则定义 power 区域 $P_i^{w_i}$ 的容量为

$$m_i = |P_i^{w_i}| = \int_{x \in P_i^{w_i}} \rho(x) dx. \quad (4)$$

各 power 区域容量的和为

$$\sum_{i=1}^n \int_{x \in P_i^{w_i}} \rho(x) dx = \int_{\Omega} \rho(x) dx. \quad (5)$$

在 3D 空间,当密度函数为常数时,各 power 区域的容量即为其体积。

给定预设容量集合 $C = \{c_i | i=1, 2, \dots, n\}$, $c_i > 0$ 且 $\sum_{i=1}^n c_i = \int_{\Omega} \rho(x) dx$,通过调整站点集 (X, W) 中的权重,使得每个区域的容量满足 $m_i = c_i$,即得到严格满足容量限制的 power 图。

在某些情况下,可能会出现站点不在所属 power 区域的情况,达不到预期的应用效果。为此,进一步对容量限制 power 图施加质心约束,使每个站点 x_i 均在其 power 区域 $P_i^{w_i}$ 的质心 b_i 内,即 3D-CCCPD。

$$x_i = b_i = \frac{\int_{x \in P_i^{w_i}} x \rho(x) dx}{\int_{x \in P_i^{w_i}} \rho(x) dx}. \quad (6)$$

2.3 JFA 算法

JFA 算法最初由 RONG 等^[23]提出,随后陆续得到了 JFA 的性质和变体 (1+JFA, JFA+1, halving

resolution 等)^[24]。因受当时硬件条件限制,仅给出了 3D-Voronoi 图的简单扩展。对 $N \times N \times N$ 的三维离散空间,首先考虑将其转化为 N 个 $N \times N$ 的二维平面,然后将各个站点映射至 N 个平面,逐层调用 JFA。JFA 无法保证每次都能生成正确的 Voronoi 图。RONG 等^[23]解释了 JFA 产生误差的原因,提出了 1+JFA 算法,并用实验验证了 1+JFA 算法可有效降低 JFA 算法的错误率。RONG 等^[24]提出了另一种 JFA 变体,即 halving resolution 算法,在相同分辨率下,其较一般 JFA 速度更快。

用 JFA 算法构造 3D-power 图的过程如图 2 所示。将每个像素 $P(x, y, z)$ 所存储的“信息”(站点坐标和权重)以步长 l 扩散至像素点 $P(x + k_1 \times l, y + k_2 \times l, z + k_3 \times l)$,其中, $k_1, k_2, k_3 \in \{-1, 0, 1\}$ 且 $l \in \{n/2, n/4, \dots, 1\}$ 。每个像素点在接收到“信息”后按式(3)计算 power 距离,并保存与 power 距离最小的站点“信息”,继续迭代,后轮迭代步长取前轮迭代步长的 $1/2, \dots$,直至 $l=1$,最终生成 3D-power 图。

图 2 中,(a)为初始状态,在 $8 \times 8 \times 8$ 的立方体中,左前下角有一个种子点。首先,以步长为 4 扩散为状态(b),然后,状态(b)以步长为 2 扩散为状态(c),最后,状态(c)以步长为 1 扩散为状态(d)。即 $N \times N \times N$ 的立方体只需经过 $\log N$ 次迭代便可将种子点扩散至整个立方体。

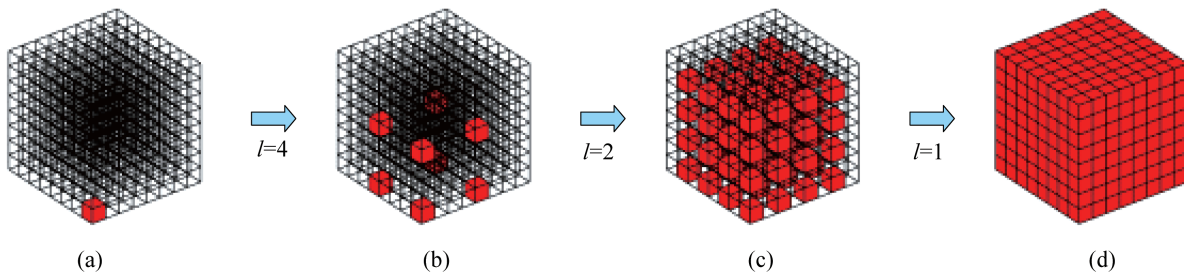


图 2 JFA 算法在 3D 空间的迭代过程

Fig.2 The iterative process of JFA algorithm in 3D space

3 基于 GPU 的 3D-CCCPD 生成算法

随着 power 图的应用领域越来越广,问题域的规模越来越大、限制条件越来越复杂、生成速度要求越来越快,尤其是 3D 空间,以 CGAL 为基础基于 CPU 的 power 图构造方法已无法满足实际应用需要。为此,本文引入 GPU,以提高 3D-power 图的构

造效率,并结合 CPU 中的 Lloyd 算法和牛顿法优化质心和容量,以大幅提升 3D-power 图的生成效率,使其能适应大规模站点等三维应用场景。

3.1 问题描述

从能量优化角度,对质心容量限制 power 图,定义能量函数 $F(X, W)$:

$$F(X, W) = \sum_{i=1}^n \int_{P_i^{w_i}} \|x - x_i\|^2 \rho(x) dx -$$

$$w_i(m_i - c_i). \quad (7)$$

文献[2-6]证明了能量函数 $F(X, W)$:

$$\begin{cases} \nabla_{w_i} F(X, W) = m - m_i, \\ \nabla_{x_i} F(X, W) = 2m_i(x_i - b_i), \\ \nabla_{w_i}^2 F(X, W) = A_{ij}/2e_{ij}. \end{cases} \quad (8)$$

本文将 JFA 变体 halving resolution 算法扩展至 3D-power 图的计算。步骤如下:

(1) 将 $N \times N \times N$ 分辨率下的初始像素图映射至 $N/2 \times N/2 \times N/2$ 分辨率下(即将 8 个像素映射在一个像素中,如果此 8 个像素中有多个站点值,则只选择其中 1 个)。

(2) 进行一般 JFA 迭代,生成 3D-power 图。

(3) 将 $N/2 \times N/2 \times N/2$ 分辨率下生成的 3D-power 图再次映射至 $N \times N \times N$ 分辨率下。

(4) 进行一次步长为 1 的 JFA 迭代,以平滑边界。

再通过 Lloyd 算法和牛顿法迭代求解能量函数 $F(X, W)$ 的最小值,加速生成 3D-CCCPD,提高 3D-power 图的适用性。

3.2 估值算法

ZHENG 等^[25]提出的连通域算法,将由 JFA 生成的 2D 平面 power 图纹理中的像素点划分为 4 种情况,据此提取 power 图的顶点,连接得到具有几何结构数据的 power 图。在 3D-power 图纹理中,很难对像素点的分布进行具体划分。即使用能够提取 3D-power 图的顶点构造三维凸包,也会严重影响时间效率。因此,需提出一种适合 3D 空间的计算 power 图几何数据的方法。

考虑 3D-power 图生成的最小计算单位为像素点,由式(8)知,在用牛顿法优化容量时较难计算相邻 power 区域相交的面积,而 power 图中边和面均由像素点构成,若能直接统计相交面上像素点的个数,并以此作为其面积的估值,则能达到优化 3D-power 图容量的目的。

图 3 中,(c)为(a)中红色标记区域平滑边界前的放大图,(d)为(b)中红色标记区域平滑边界后的放大图。观察到在 JFA 变体 halving resolution 算法中,最后一步以步长为 1 迭代平滑边界,只计算了 power 图“边界”(2D-power 图中为边,3D-power 图中为面)像素点。所以,对平滑边界,只需对这部分像素点进行标记,统计其数量并作为当前 power 区域的面积估值。在平滑边界的同时计算 power 图的几何数据,以提高时间性能。

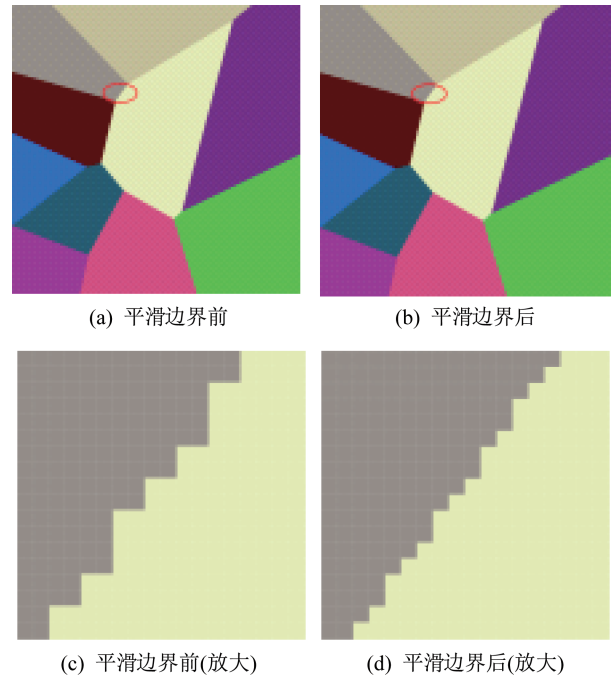


图3 JFA 平滑边界

Fig.3 Smooth boundary of JFA

以图 4 的估值算法为例,在问题域内有红色(x_r),绿色(x_g)和蓝色(x_b) 3 个站点,经 JFA 和 halving resolution 算法迭代,得到如图 4 所示的结果。在像素点 P_1 内存储有 x_b 的信息,在像素点 P_2 内存储有 x_r 的信息。当进行步长 $l=1$ 的迭代时,经计算, $d_{b1} < d_{r1}$, 像素点 P_1 内存储的信息不变。而对于像素点 P_2 ,经计算, $d_{b2} < d_{r2}$, 像素点 P_2 内的信息更新为 x_b ,同时将站点 x_r 和 x_b 的公共面 A_{rb} 的统计数组 $\text{count}[r][b]$ 加 1。

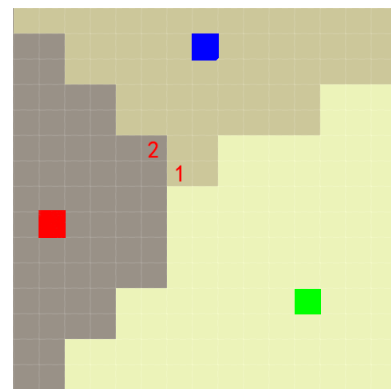


图4 估值算法

Fig.4 Estimating algorithm

算法 1 估值算法

输入 3D-power 图和站点数 n

输出 3D-power 图和 A_{ij}

Step 1 初始化统计数组 $\text{count}[n][n]$

Step 2 将保存在像素点 $P(x, y, z)$ 中的站点 ID 扩散至 $P(x + k_1, y + k_2, z + k_3)$, $k_1, k_2, k_3 \in \{-1, 0, 1\}$

Step 3 由式(3), 计算 power 距离 d_i 和 d_j

Step 4 如果 $d_j < d_i$, 则

Step 4.1 $\text{count}[i][j]++$

Step 4.2 更新站点中保存的“信息”

Step 5 结束

Step 6 //统计, 取同一个面估值的均值

$A_{ij} = (\text{count}[i][j] + \text{count}[j][i]) / 2$

Step 7 输出: 3D-power 图及 A_{ij}

3.3 3D-power 图构造算法

用 1+JFA 和 halving resolution 算法构造 3D-power 图。在构造过程中调用估值算法, 计算每个 power 区域的面积 A_{ij} 。

算法 2 基于 GPU 的 3D-power 图构造算法

输入 站点集: $X = \{x_i, i = 1, 2, \dots, n\}$,

权重集: $W = \{w_i, i = 1, 2, \dots, n\}$

输出 3D-power 图

Step 1 //初始化 GPU 纹理, 为每个站点分配唯一 ID $\in \{1, 2, \dots, n\}$

Step 2 重复

Step 2.1 若当前像素点中存在站点, 则

Step 2.1.1 将站点 ID 保存在此像素点中

Step 2.2 且

Step 2.2.1 像素点中保存“-1”

Step 2.3 结束

Step 3 直到所有像素点完成初始化

Step 4 将 3D 纹理分辨率减半至 $N = N/2$

Step 5 调用 JFA_Kernel(), $l = 1$

Step 6 对 $l = N/2; l \geq 1; l = l/2$

Step 6.1 调用 JFA_Kernel(), l

Step 7 结束

Step 8 将得到的纹理映射到 $N = 2N$ 的 3D 纹理中

Step 9 调用算法 1//估值算法

Step 10 输出渲染后的 3D-power 图

Step 11 ///子程序 JFA_Kernel()

Step 12 //GPU 并行地将“信息”传播至每个像素, 为每个站点分配唯一 ID

Step 13 //26 个传播方向, 将保存在像素点 $P(x, y, z)$ 中的站点 x_i 的 ID 扩散到 $P(x + k_1 \times l, y + k_2 \times l, z + k_3 \times l)$, $k_1, k_2, k_3 \in \{-1, 0, 1\}$

Step 14 由式(3), 计算 power 距离 d_i 与 d_j

Step 15 若 $d_j < d_i$, 则

Step 15.1 将 $P(x + k_1 \times l, y + k_2 \times l, z + k_3 \times l)$ 中的“信息”更新为站点 x_j 的 ID

Step 16 结束

3.4 3D-CCCPD 算法

3D-CCCPD 的容量优化算法——牛顿法, 其海森矩阵的数据来源于 GPU 端的估值算法, 但牛顿法迭代步长的计算运行于 CPU 端。由于涉及矩阵求逆和矩阵相乘, 计算较为耗时。

算法 3 3D-power 图优化生成算法

输入 站点集: $X = \{x_i, i = 1, 2, \dots, n\}$

权重集: $W = \{w_i, i = 1, 2, \dots, n\}$

容量集: $C = \{c_i, i = 1, 2, \dots, n\}$

容量精度停止条件: t_w

质心精度停止条件: t_x

输出 3D-CCCPD

Step 1 重复

Step 1.1 GPU: 调用算法 2 构建 3D-power 图

Step 1.2 重复

Step 1.2.1 //牛顿法优化容量

CPU: 计算牛顿法迭代步长 δw

Step 1.2.2 $W \leftarrow W + \delta w$

Step 1.2.3 GPU: 调用算法 2 构造 3D-power 图

Step 1.3 直到 $\|\nabla_w F\| \leq t_w$

Step 1.4 //GPU-Lloyd 算法优化质心

GPU: 并行计算每个区域的质心 b_i

Step 1.5 $X \leftarrow b$

Step 2 直到 $\|\nabla_x F\| \leq t_x$

Step 3 输出 3D-CCCPD

4 实验分析

实验环境为 Microsoft Windows 10, Intel(R) Core(TM) i7-4720 CPU @ 2.60 GHz, 16.0 GB RAM。编译环境为 Microsoft Visual C++ 2012, GPU 为 NVIDIA GeForce GTX 960M, GPU 并行计算平台为 CUDA 8.0。实验结果用 MATLAB R2019b 渲染。

4.1 分辨率设置

图 5 为不同分辨率及不同站点数下用算法 2 构造 3D-power 图的时间性能对比。可见, 在相同分辨率、不同站点数下, 算法 2 的时间性能相对稳定; 在相同站点数下, 构造 3D-power 图的时间随分辨率的增加而增长。因此需选取一个合适的分辨率。由图 5, 综合时间性能和准确率, 选取分辨率 $N = 256$ 的正方体作为问题域。

4.2 实验结果

设质心精度 $t_x = 2$, 容量精度 $t_w = 0.0003$ 。图 6 为生成的 3D-CCCPD 立体图, 第 1 列展示的为站

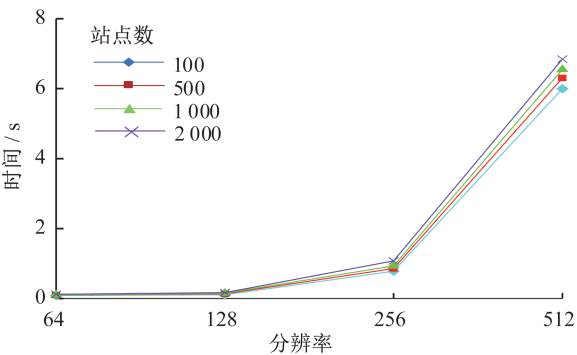


图5 其于GPU构造3D-power图的算法性能

Fig. 5 The computational performance of the 3D-power diagram constructing algorithm based on GPU

点数分别为1 000,2 000,5 000时的3D-CCCPD立体图效果,第2列展示的为站点数分别为1 000,2 000,5 000时的3D-CCCPD透视图,第3~5列为Z坐标轴逐层拆解得到的X-Y平面图,从左到右分别为第20,80和160层(共N=256层平面图)。可知,本文算法生成的3D-CCCPD其结构较均匀紧凑。

4.3 算法性能

表1为不同站点数下,本文算法2生成power图的时间性能对比。可知,在不同数量级的站点数下,算法2生成power图的时间性能相对稳定,适用于较多站点情况下3D-power图的生成。

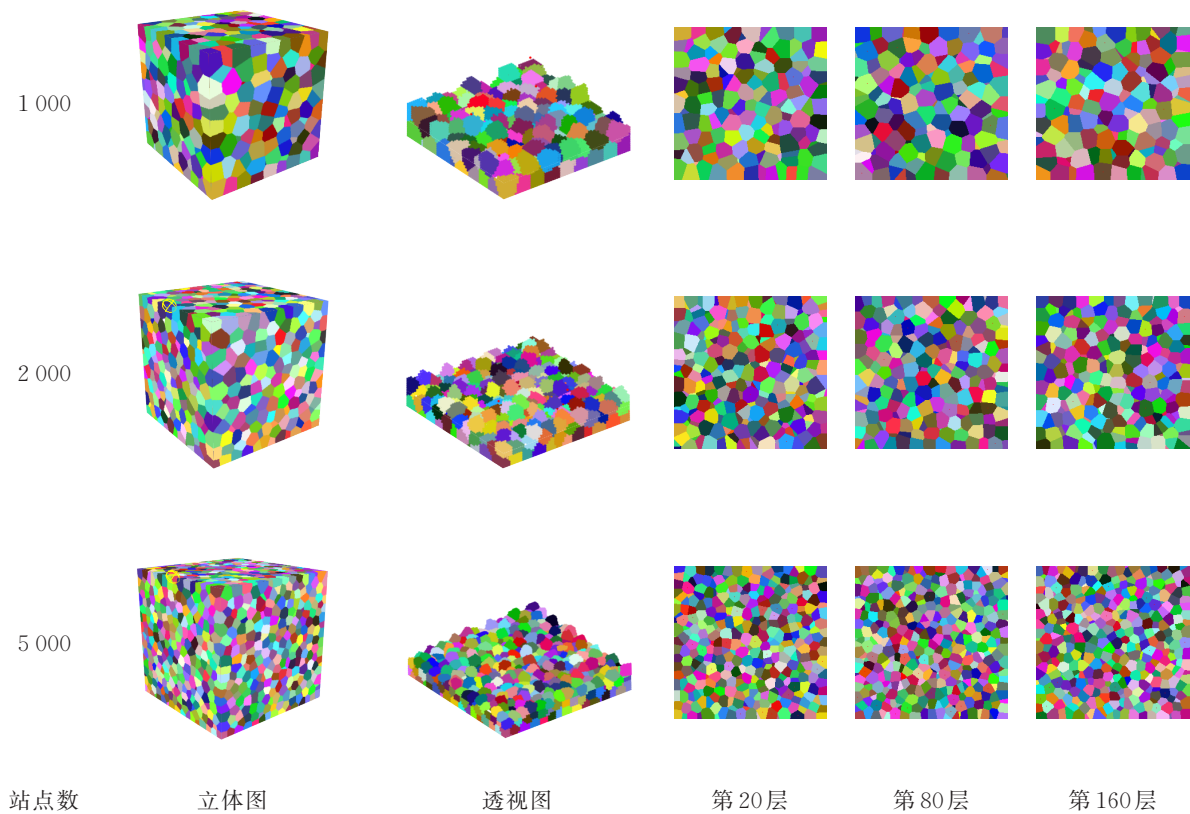


图6 3D-CCCPD立体图及截面图

Fig.6 The stereogram and cross-sectional views of 3D-CCCPD

表2给出了以CGAL为基础的算法与本文算法3及生成3D-CCCPD的时间性能对比。其中, T_{CGAL} 为以CGAL为基础的算法生成3D-CCCPD的时间性能, T_{ours} 为本文算法3生成3D-CCCPD的时间性能,容量精度停止条件为 $t_w=0.0003$,质心精度停止条件为 $t_r=2$,即2个像素点的距离。定义CGAL与本文算法3的加速比(sp)为

$$sp=\frac{T_{CGAL}}{T_{ours}},\tag{9}$$

当站点数由2 000增至5 000时,加速比下降,这是因为本文算法3在站点数为5 000时,时间消耗发生了突增,其中,在基于CPU平台的牛顿法优化过程中,迭代步长的计算时间突然增加,其受站点数影响较大。另外,本文算法3的时间消耗随站点数增加而增加,较以CGAL为基础的算法优势明显,且随着站点数的增加,加速效果更明显。

表1 不同站点数下本文算法2生成3D-power图的时间性能

Table 1 Computational performance of 3D-power diagrams construction of algorithm 2 with different number of sites

序号	站点数	耗时/s
1	100	0.782
2	500	0.857
3	1 000	0.933
4	2 000	1.078
5	5 000	1.814

表2 不同站点数下以CGAL为基础的算法与本文算法3的时间性能对比

Table 2 Comparison results of algorithm based on CGAL and the proposed algorithm 3 with different number of sites

序号	站点数	CGAL 耗时/s	本文算法 耗时/s	加速比
1	1 000	517.027	5.838	88.562
2	2 000	1 749.329	7.819	223.728
3	5 000	6 997.011	44.519	157.169

4 结 论

将JFA应用于3D-power图的构造,提出了一种牛顿法优化3D-CCCPD的容量估值算法,通过与JFA变体 halving resolution 算法结合,极大提高了生成3D-power图的时间性能,结合最优化算法,能够快速生成满足约束条件的3D-CCCPD。

实验中发现,算法3的牛顿法迭代步长计算对最终的时间性能影响较大。下一步将继续研究纯GPU平台下生成3D-CCCPD的算法,以及在保证时间性能的同时生成更高精度的3D-CCCPD。

参考文献(References):

[1] BALZER M, SCHLÖMER T, DEUSSEN O. Capacity-constrained point distributions: A variant of Lloyd's method[J]. **ACM Transactions on Graphics**, 2009, 28(3): 1-8. DOI: 10.1145/1531326. 1531392

[2] GOES F de, BREEDEN K, OSTROMOUKHOV V, et al. Blue noise through optimal transport[J]. **ACM Transactions on Graphics**, 2012, 31(6): 1-11. DOI: 10.1145/2366145.2366190

[3] ZHANG S, GUO J, ZHANG H, et al. Capacity constrained blue-noise sampling on surfaces [J]. **Computers & Graphics**, 2016, 55: 44-54. DOI: 10.1016/j.cag.2015.11.002

[4] AHMED A G M, GUO J, YAN D M, et al. A simple push-pull algorithm for blue-noise sampling [J]. **IEEE Transactions on Visualization and Computer Graphics**, 2017, 23(12): 2496-2508. DOI: 10.1109/TVCG.2016.2641963

[5] 郑利平,程亚军,周乘龙,等. 异构群体队形光滑变换控制方法[J]. **计算机辅助设计与图形学学报**, 2015, 27(10): 1963-1970. DOI: 10.3969/j. issn. 1003-9775. 2015.10.020

ZHENG L P, CHENG Y J, ZHOU C L, et al. Research on smooth formation control of heterogeneous crowds [J]. **Journal of Computer-Aided Design & Computer Graphics**, 2015(10): 1963-1970. DOI: 10.3969/j. issn. 1003-9775.2015. 10.020

[6] GOES F de, WALLEZ C, HUANG J, et al. Power particles: An incompressible fluid solver based on power diagrams [J]. **ACM Transactions on Graphics**, 2015, 34(4): 1-11. DOI: 10.1145/2766901

[7] AANJANEYA M, GAO M, LIU H, et al. Power diagrams and sparse paged grids for high resolution adaptive liquids[J]. **ACM Transactions on Graphics**, 2017, 36(4): 1-12. DOI: 10.1145/3072959.3073625

[8] ZHAI X, HOU F, QIN H, et al. Fluid simulation with adaptive staggered power particles on GPUs [J]. **IEEE Transactions on Visualization and Computer Graphics**, 2018, 26(6): 2234-2246. DOI: 10.1109/TVCG.2018.2886322

[9] AURENHAMMER F. Power diagrams: Properties, algorithms and applications [J]. **SIAM Journal on Computing**, 1987, 16: 78-96. DOI: 10.1137/0216006

[10] AURENHAMMER F. Voronoi diagrams: A survey of a fundamental geometric data structure [J]. **ACM Computing Surveys**, 1991, 23(3): 345-405. DOI: 10.1145/116873.116880

[11] BALZER M, HECK D. Capacity-constrained Voronoi diagrams in finite spaces[C]// **Proceedings of the 5th International Symposium on Voronoi Diagrams in Science and Engineering**. Kiev, Ukraine: IEEE, 2008: 44-56.

[12] BALZER M. Capacity-constrained Voronoi diagrams in continuous spaces [C]// **Proceedings of the 6th International Symposium on Voronoi Diagrams**. Copenhagen: IEEE, 2009: 79-88. DOI: 10.1109/isvd.

- 2009.28
- [13] MULLEN P, MEMARI P, GOES F de, et al. HODGE-optimized triangulations [J]. **ACM Transactions on Graphics**, 2011, 30(4): 1-11. DOI: 10.1145/2010324.1964998
- [14] XIN S Q, LÉVY B, CHEN Z, et al. Centroidal power diagrams with capacity constraints: Computation, applications, and extension[J]. **ACM Transactions on Graphics**, 2016, 35(6): 1-12. DOI: 10.1145/2980179.2982428
- [15] LIU X, MA L, GUO J, et al. Parallel computation of 3D clipped Voronoi diagrams [C]// **IEEE Transactions on Visualization and Computer Graphics**. Stony Brook: IEEE, 2020. DOI: 10.1109/TVCG.2020.3012288
- [16] HAN J, YAN D M, WANG L, et al. Computing restricted Voronoi diagram on graphics hardware[C]// **Proceedings of the 25th Pacific Conference on Computer Graphics and Applications**. Maui: IEEE Computer Society, 2017: 23-26.
- [17] YAN D M, WANG W, LÉVY B, et al. Efficient computation of 3D clipped Voronoi diagram [C]// HUTCHISON D, KANADE T, KITTLER J, et al. **Advances in Geometric Modeling and Processing**. Berlin/Heidelberg: Springer, 2010: 269-282. DOI: 10.1007/978-3-642-13411-1_18
- [18] ALLIZE P, FABRI A. Computational geometry algorithms library [C]// **Proceedings of the 2009 ACM SIGGRAPH ASIA**. New York: ACM, 2009: 1-133. DOI: 10.1145/1665817.1665821
- [19] RAY N, SOKOLOV D, LEFEBVRE S, et al. Meshless Voronoi on the GPU [J]. **ACM Transactions on Graphics**, 2018, 37(6): 1-12. DOI: 10.1145/3272127.3275092
- [20] LIU X, YAN D M. Computing 3D clipped Voronoi diagrams on GPU [C]// **SIGGRAPH Asia 2019 Posters**. New York: ACM, 2019: 1-2. DOI: 10.1145/3355056.3364581
- [21] RYCROFT C H. VORO++: A three-dimensional Voronoi cell library in C++ [J]. **Chaos**, 2009, 19(4): 9041111. DOI: 10.1063/1.3215722
- [22] GOLIN M J, NA H S. On the average complexity of 3D-Voronoi diagrams of random points on convex polytopes [J]. **Computational Geometry**, 2003, 25(3): 197-231. DOI: 10.1016/S0925-7721(02)00123-2
- [23] RONG G D, TAN T S. Jump flooding in GPU with applications to Voronoi diagram and distance transform [C]// **Proceeding of the 2006 Symposium on Interactive 3D Graphics and Games**. Redwood City, CA: ACM, 2006: 109-116. DOI: 10.1145/1111411.1111431
- [24] RONG G, TAN T S. Variants of jump flooding algorithm for computing discrete Voronoi diagrams [C]// **The 4th International Symposium on Voronoi Diagrams in Science and Engineering**. Glamorgan: IEEE Computer Society, 2007: 176-181. DOI: 10.1109/isvd.2007.41
- [25] ZHENG L, GUI Z, CAI R, et al. GPU-based efficient computation of Power diagram [J]. **Computers & Graphics**, 2019, 80: 29-36. DOI: 10.1016/j.ag.2019.03.011